FlowMemory External Developer Review Packet

External Developer Review Packet

This packet is written for a senior developer or security reviewer who needs to understand the project quickly, challenge it hard, and separate real technical claims from marketing language.

Copy-Paste Note To The Reviewer

I am asking you to review this repository as a technical and security skeptic.

The repo is:

```
https://github.com/FlowmemoryAI/flowmemory-uniswap-v4-hooks
```

The core claim is narrow:

```
The swap is not the memory.
The transaction is the proof envelope.
The FlowPulse is the memory artifact.
```

Please do not review this as a normal swap hook, fee hook, routing hook, or custody system. The intended primitive is a memory-signal hook: a Uniswap v4 `afterSwap` boundary that emits a structured `FlowPulse` event after a completed swap boundary. Downstream reader/verifier tooling attaches receipt metadata later and checks whether later machine-memory history is consistent with that public boundary.

I want you to look for technical weakness, overclaiming, security problems, fake novelty, unverifiable assumptions, and anything that sounds stronger than the code supports.

Simple Explanation

Most DeFi systems already have execution:

- swaps;
- settlement;
- liquidity;
- fees;
- routing;
- accounting;
- transaction logs.

FlowMemory is trying to add a missing layer: memory.

The project does not say that a swap itself is memory. It says that the completed swap boundary can emit a memory signal. That signal is called a `FlowPulse`.

The transaction is the proof envelope because the receipt proves where the event landed. The `FlowPulse` is the memory artifact because it is the structured signal downstream systems can cite, check, and compose.

The simplest mental model:

```
Uniswap v4 swap happens
  -> PoolManager reaches afterSwap
  -> FlowMemory hook emits FlowPulse
  -> transaction receipt proves the event landed
  -> reader attaches txHash/logIndex after the fact
  -> downstream systems can test whether later memory claims are possible
```

Technical Summary

This repository has two connected layers.

First, it implements a narrow Uniswap v4 `afterSwap` hook:

- `contracts/FlowMemoryAfterSwapHook.sol`
- `contracts/FlowPulse.sol`
- `contracts/FlowMemoryHookPlanner.sol`
- `contracts/interfaces/IFlowMemoryHookData.sol`
- `contracts/interfaces/IUniswapV4SwapHookLike.sol`
- `test/FlowMemoryAfterSwapHook.t.sol`

Second, it builds local R&D tooling around memory consistency:

- FMM-0: a receipt-bound memory consistency model for machine histories.
- FlowLitmus: executable forbidden-outcome tests.
- FlowSerial: receipt-linearizable machine-history checks.
- Cache Lineage Gate: proof-carried KV/context reuse checks.
- Compute Reuse Router: proof-backed AI/GPU-workflow reuse decisions.
- Compute Reuse Consistency: cache, compute, and receipt-history reuse harness.
- Public Claim Gate: a deterministic check against public overclaims.

The project is launch-prep and public R&D. It is not claiming live Base mainnet deployment, audited custody, production verifier readiness, GPU hardware speedup, semantic truth, or model correctness.

Architecture In One Diagram

```
Execution Layer
  Uniswap v4 PoolManager
  swap lifecycle
  afterSwap boundary

Memory Emission Layer
  FlowMemoryAfterSwapHook
  FlowPulse
  rootfieldId
  commitment
  parentPulseId
  uri

Evidence Layer
  transaction receipt
  txHash
  transactionIndex
  logIndex
  finality policy
  reader-derived metadata

Memory Layer
  FMM-0
  FlowSerial
  FlowLitmus
  FlowMemory / Rootflow downstream state
  cache and compute reuse gates
```

Important separation:

```
The hook emits the memory signal.
The reader proves where it landed.
The memory model checks whether later history could have happened.
```

Solidity Boundary

The hook is deliberately minimal. That is a security feature, not a weakness. It is not trying to change swap execution.

Expected invariants:

- only `afterSwap` is intended;
- callback is gated to the configured PoolManager;
- `hookData` is required;
- `rootfieldId` is required;
- `commitment` is required;

- `sender` is required;
- the hook returns the correct `afterSwap` selector;
- the hook returns zero hook delta;
- no token custody path exists;
- no dynamic fee path exists;
- no routing path exists;
- no custom accounting path exists;
- no hook-time `txHash`, `transactionIndex`, or `logIndex` exists in the event;
- URI content is advisory and untrusted;
- commitment is opaque;
- actor may be a router or contract sender, not necessarily the end-user EOA.

The security thesis is:

```
The hook should do one thing: emit the memory signal at a verified boundary.
```

Event And Evidence Model

`FlowPulse` is the memory artifact emitted by the contract.

The event intentionally includes:

- `pulseId`
- `rootfieldId`
- `actor`
- `pulseType`
- `subject`
- `commitment`
- `parentPulseId`
- `sequence`
- `occurredAt`
- `uri`

The event intentionally excludes:

- `txHash`

- `transactionIndex`
- `logIndex`

Those receipt facts cannot be known by the hook during execution. They are attached later by reader/verifier infrastructure from the transaction receipt.

This is one of the most important review points. If the repo ever implies that the hook itself knows receipt metadata during execution, that would be wrong.

What This Actually Proves Today

The current repo proves a local launch-prep surface:

- the Solidity hook emits a `FlowPulse` and returns zero hook delta;
- unauthorized callers cannot invoke the hook callback directly;
- invalid hook data is rejected;
- event schemas exclude hook-time receipt metadata assumptions;
- the planner constrains the hook permission surface;
- local FMM-0 tooling catches impossible machine histories;
- cache and compute reuse are blocked when lineage, runtime, attestation policy, or receipt-bound history drift;
- public launch language is checked for unguarded overclaims.

It does not yet prove a public live deployment. Public Base Sepolia receipt evidence remains pending until a release packet exists with deployment transaction, observed logs, reader evidence, and receipt-derived metadata.

What This Does Not Claim

Do not credit the project for these until separate evidence exists:

- live Base mainnet deployment;
- audited custody infrastructure;
- fund protection;
- swap control;
- fee control;
- routing control;

- custom accounting;
- semantic truth;
- model correctness;
- GPU hardware speedup;
- production verifier readiness;
- hook-time knowledge of `txHash`, `transactionIndex`, or `logIndex`;
- every ordinary Uniswap transaction automatically becoming FlowMemory.

Why The Idea Is Different

Most Uniswap v4 hooks ask:

```
How can we change what the swap does?
```

FlowMemory asks:

```
What should the system remember once execution has happened?
```

That is the category difference. The hook does not optimize the trade, route the swap, custody funds, or adjust accounting. It emits a memory artifact at a boundary that the chain can prove happened.

Compared with ordinary indexers, FlowMemory is not merely scraping history after the fact. It emits an intentional memory signal at the execution boundary.

Compared with ordinary AI memory, FlowMemory is not just retrieving context. It asks whether a claimed machine history could have happened around receipt-bound events.

Compared with GPU memory optimizations, FlowMemory is not making chips faster. It is making unsafe compute reuse harder to get wrong.

The better-than-normal axis is specific:

```
FlowMemory gives machine memory an external execution boundary and executable
consistency tests.
```

How To Verify Locally

Run this first:

```
python tools/flowmemory_release_transcript.py --pretty
```

Expected high-level status:

```
Local FMM-0 consistency surface: PASS
Public Base Sepolia receipt evidence: PENDING
Production verifier infrastructure: NOT_CLAIMED
```

Run the core verification set:

```
python -m unittest discover -s tools -p 'test_*.py'
forge fmt --check
forge build
forge test -vvv
python tools/public_claim_gate.py --pretty
python tools/launch_reality_check.py --pretty
python tools/compute_reuse_consistency.py demo --pretty
```

Expected current local results:

- Python/tool tests: 276 passed.
- Foundry tests: 12 passed.
- Public Claim Gate: 0 unguarded overclaims.
- Launch Reality Check: PASS.
- Compute Reuse Consistency: 5/5 cases passed, 4/4 unsafe reuse blocked.

Suggested 60-Minute Review Path

1. Read `README.md` for the claim surface.
2. Read `contracts/FlowMemoryAfterSwapHook.sol`.
3. Read `contracts/FlowPulse.sol`.
4. Read `test/FlowMemoryAfterSwapHook.t.sol`.
5. Run `forge test -vvv`.
6. Run `python tools/launch_reality_check.py --pretty`.
7. Run `python tools/public_claim_gate.py --pretty`.
8. Read `docs/SKEPTIC_REVIEW_WALKTHROUGH.md`.
9. Read `FLOWMEMORY_PUBLIC_TECHNICAL_REPORT.md` or the PDF.
10. Review the pending public release evidence path in `releases/base-sepolia/`.

Security Review Checklist

Please review these areas first:

- PoolManager gating: can any non-PoolManager caller fabricate hook callbacks?
- Hook permission bits: does the planned address match only the intended `afterSwap` permission?
- Return values: does the hook always return the expected selector and zero hook delta on success?
- Input validation: are empty `hookData`, zero `rootfieldId`, zero `commitment`, and zero `sender` rejected?
- Event schema: does `FlowPulse` avoid impossible hook-time receipt metadata?
- State mutation: is the only stateful behavior sequence/pulse derivation, not funds, balances, fees, or routing?
- URI handling: is URI treated as advisory and untrusted?
- Commitment handling: is the commitment treated as opaque rather than as a semantic truth oracle?
- Actor semantics: does the code avoid promising actor equals end-user EOA?
- Deployment assumptions: are Base Sepolia and mainnet claims kept separate?
- Reader trust: is reader-derived metadata clearly post-execution?
- Public copy: does the repo avoid claiming more than the code proves?

Questions That Should Be Used To Try To Break It

Ask these directly:

1. Can the hook custody, move, lock, or protect funds?
2. Can the hook change fees, routing, swap execution, or pool accounting?
3. Can any caller besides the configured PoolManager call the hook successfully?
4. Does the hook ever claim to know `txHash` or `logIndex` during execution?
5. Does `uri` prove the truth of the memory content?
6. Does `commitment` prove semantic truth, or only bind to an opaque artifact?
7. Does the repo claim live Base mainnet deployment?
8. Does the repo claim an audit?
9. Does the repo claim production verifier readiness?

10. Does the GPU/AI section claim hardware speedup, or only reuse discipline?
11. Can the tests be reduced to ordinary event emission with marketing around it, or is the FMM-0 forbidden-outcome surface doing real consistency work?
12. What would be needed before this should be trusted in production?

Honest Risk Register

The largest risks are not hidden:

- Public deployment evidence is pending.
- Production verifier infrastructure is not claimed.
- The current package is not an audit.
- The memory model is local R&D, not a finalized standard.
- URI content can lie.
- Commitments bind to artifacts but do not prove semantic truth by themselves.
- Reader/verifier bugs could attach incorrect receipt metadata.
- Deployment misconfiguration could point the hook at the wrong PoolManager.
- Category language can overrun the evidence if the claim gate is bypassed.

What Would Make It Stronger

A senior reviewer should expect the next maturity steps to include:

- public Base Sepolia release evidence packet;
- verified contract source on explorer;
- observed `FlowPulse` logs from real hook activity;
- reader output that attaches `txHash`, `transactionIndex`, and `logIndex`;
- gas snapshots and deployment reproducibility notes;
- independent security review;
- fuzz/property tests around hook data decoding and sequence behavior;
- richer examples showing downstream FlowMemory / Rootflow use;
- formalized FMM-0 spec maturity path;
- production-ready verifier design work with explicit trust assumptions.

Bottom Line

The project should be judged on this claim:

```
FlowMemory uses a Uniswap v4 afterSwap boundary to emit a FlowPulse memory
signal, then uses receipt-bound consistency tooling to reject impossible
machine-memory and compute-reuse histories.
```

That is meaningful if the code and tools preserve the boundary:

```
swap != memory
transaction = proof envelope
FlowPulse = memory artifact
```

If that boundary is technically clean, the repo is a credible first public step toward memory-native DeFi infrastructure. If that boundary is blurred, the category claim should be rejected.